รายงานโครงงานที่ 1 · วิชา 040613204

การเขียนโปรแกรมเชิงวัตถุ (Object-Oriented Programming)

ยันต์ล้อมวัด

YANTRA WARD : TEMPLE DEFENSE



จัดทำโดย	นายกฤตภาส ชูภักดี
รหัสนักศึกษา	6804062612293
เสนอ	ผู้ช่วยศาสตราจารย์สถิตย์ ประสมพันธ์
ภาคการศึกษา	ภาคเรียนที่ 1 ปีการศึกษา 2569

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์

มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

https://github.com/Krittapas-01/YantraWard

สารบัญ

บทที่ 1 บทนำ **3**

1.1 ที่มาและความสำคัญ 3

1.2 วัตถุประสงค์ 3

1.3 ขอบเขตของโครงงาน 4

1.4 เครื่องมือที่ใช้ 4

บทที่ 2 รายละเอียดของเกม **5**

2.1 เนื้อเรื่องและเป้าหมาย 5

2.2 วิธีควบคุม 6

2.3 เครื่องราง (ป้อม) 6

2.4 วิญญาณ (ศัตรู) 7

2.5 ด่านและระดับความยาก 8

บทที่ 3 การออกแบบโปรแกรม **10**

3.1 โครงสร้างแพ็กเกจ 10

3.2 แผนภาพคลาส 10

3.3 วงจรการทำงานของเกม (Game Loop) 11

บทที่ 4 หลักการเขียนโปรแกรมเชิงวัตถุที่ใช้ **13**

4.1 Encapsulation และ Custom Exception 13

4.2 Inheritance และ Abstraction 14

4.3 Polymorphism 15

4.4 Interface และ Default Method 16

4.5 Template Method 16

4.6 Enum แบบ Factory, Generics และ Overloading 17

4.7 Thread และ File I/O 18

บทที่ 5 อัลกอริทึมและเทคนิคที่น่าสนใจ **19**

5.1 หาเส้นทางด้วย BFS 19
5.2 การเล็งดักหน้าของน้ำมนต์ 19
5.3 สังเคราะห์เสียงด้วยโค้ด 20
บทที่ 6 การทดสอบและปรับสมดุลเกม 21
6.1 วิธีทดสอบด้วยบอท 21
6.2 ผลการทดสอบ 21
6.3 สิ่งที่ปรับจากผลการทดสอบ 22
บทที่ 7 ปัญหาที่พบและแนวทางแก้ไข 23
บทที่ 8 สรุปผล 24
8.1 สรุปผลการดำเนินงาน 24
8.2 สิ่งที่ได้เรียนรู้ 24
8.3 แนวทางพัฒนาต่อ 24
ภาคผนวก วิธีติดตั้งและรันเกม 25

บทที่ 1 บทนำ

1.1 ที่มาและความสำคัญ

เกมแนว Tower Defense เป็นเกมที่เหมาะกับการฝึกออกแบบเชิงวัตถุเป็นอย่างมาก เพราะในเกมมี “วัตถุ” หลายชนิดที่มีลักษณะร่วมกันแต่พฤติกรรมต่างกัน เช่น ป้อมหลายแบบที่โจมตีคนละวิธี และศัตรูหลายแบบที่เคลื่อนที่คนละวิธี ซึ่งตรงกับหลักการ Inheritance และ Polymorphism โดยตรง ผู้จัดทำจึงเลือกพัฒนาเกม **“ยันต์ล้อมวัด (Yantra Ward : Temple Defense)”** เกมป้องกันวัดจากผีไทย โดยใช้ของศักดิ์สิทธิ์แบบไทย ได้แก่ ตะกรุด ระฆัง สายสิญจน์ และน้ำมนต์ เป็นป้อมป้องกัน เพื่อให้เกมมีเอกลักษณ์และสื่อถึงวัฒนธรรมไทย

เกมนี้เขียนด้วยภาษา Java และไลบรารี Swing ล้วน ไม่ใช้ Game Engine ไม่ใช้ไลบรารีภายนอก และไม่มีไฟล์รูปภาพหรือไฟล์เสียงเลย ภาพทุกอย่างวาดด้วยคำสั่ง `Graphics2D` ส่วนเสียงเอฟเฟกต์และเพลงประกอบสังเคราะห์ขึ้นจากโค้ด ทำให้ทุกส่วนของเกมเป็นผลงานจากการเขียนโปรแกรมทั้งหมด

1.2 วัตถุประสงค์

1. เพื่อนำหลักการเขียนโปรแกรมเชิงวัตถุ (Encapsulation, Inheritance, Polymorphism, Abstraction) มาใช้กับงานจริงที่มีขนาดใหญ่กว่าแบบฝึกหัด
2. เพื่อฝึกออกแบบโครงสร้างคลาสให้ขยายต่อได้ง่าย เช่น เพิ่มป้อม ศัตรู หรือด่านใหม่โดยแก้โค้ดเดิมน้อยที่สุด
3. เพื่อฝึกใช้ความสามารถอื่นของภาษา Java ได้แก่ Interface, Enum, Generics, Exception, Thread และ File I/O
4. เพื่อสร้างเกมที่เล่นได้จริง สนุก และมีความท้าทายเหมาะกับผู้เล่นหลายระดับ

1.3 ขอบเขตของโครงงาน

- เกม 2 มิติ มุมมองจากด้านบน หน้าต่างขนาด 800 × 604 พิกเซล สนามแบบตาราง 20 × 12 ช่อง
- มี 3 ด่าน 4 ระดับความยาก (ง่าย ปกติ ยาก ฝันร้าย) ด่านละ 15 คลื่น และมีบอสในคลื่นสุดท้าย
- ป้อม 4 ชนิด อัปเกรดได้ 3 ระดับ และขายคืนได้ ศัตรู 5 ชนิด รวมบอส
- บันทึกคะแนนสูงสุดแยกตามด่านและความยาก และบันทึกการตั้งค่าเสียงลงไฟล์
- รันได้ทั้งจาก NetBeans (โปรเจกต์ Maven) และจากไฟล์ `YantraWard.jar` บน Java 8 ขึ้นไป

1.4 เครื่องมือที่ใช้

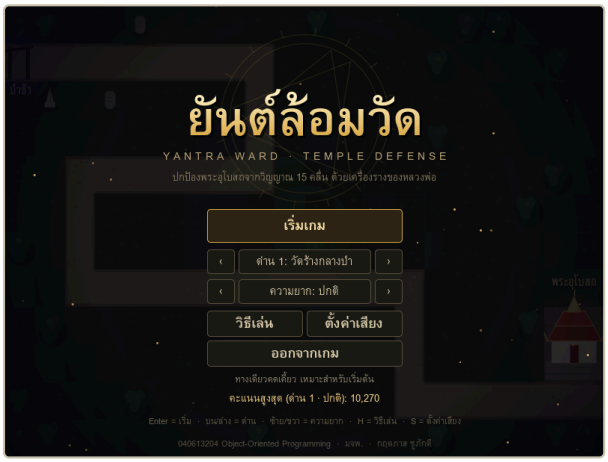
ตารางที่ 1 เครื่องมือและเทคโนโลยี

รายการ	รายละเอียด
ภาษา	Java 17 (ซอร์สโค้ด) · ไฟล์ jar คอมไพล์แบบ `--release 8` ให้รันได้ตั้งแต่ Java 8
IDE / Build	Apache NetBeans · Maven (กำหนด `exec.mainClass = yantra.Main`)
กราฟิก	Java Swing (`JFrame`, `JPanel`, `Graphics2D`) วาดเองทั้งหมด
เสียง	`javax.sound.sampled` สังเคราะห์คลื่นเสียงเป็น PCM แล้วเล่นผ่าน `Clip`
จัดการซอร์สโค้ด	Git และ GitHub — https://github.com/Krittapas-01/YantraWard

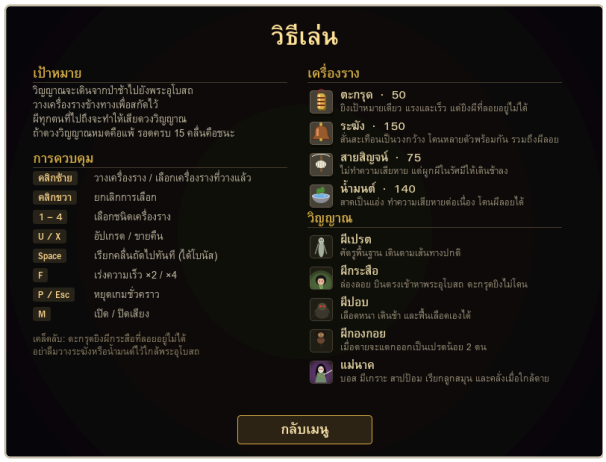
บทที่ 2 รายละเอียดของเกม

2.1 เนื้อเรื่องและเป้าหมาย

ในคืนเดือนมืด วิญญาณจากป่าช้าออกเดินตามทางมุ่งหน้าเข้าสู่พระอุโบสถ ผู้เล่นต้องวางเครื่องรางศักดิ์สิทธิ์ไว้ริมทางเพื่อปัดเป่าวิญญาณ ทุกครั้งที่วิญญาณเดินเข้าถึงพระอุโบสถ ผู้เล่นจะเสีย “ดวงวิญญาณ” 1 ดวง (เหมือนพลังชีวิต) ถ้าดวงวิญญาณหมดจะแพ้ ถ้ารอดครบ 15 คลื่นจะชนะ ทุกครั้งที่ปราบวิญญาณได้จะได้รับ “แรงศรัทธา” ซึ่งเป็นเงินในเกม ใช้ซื้อและอัปเกรดเครื่องราง



ภาพที่ 1 หน้าเมนูหลัก เลือกด่านและระดับความยาก



ภาพที่ 2 หน้าวิธีเล่น

2.2 วิธีควบคุม

ตารางที่ 2 ปุ่มควบคุม

ปุ่ม	การทำงาน
คลิกซ้าย	วางเครื่องราง / เลือกเครื่องรางที่วางแล้วเพื่อดูข้อมูล อัปเกรด หรือขาย
คลิกขวา	ยกเลิกการเลือก
1 – 4	เลือกชนิดเครื่องราง (กดซ้ำเพื่อดูรายละเอียดก่อนซื้อ)
U / X	อัปเกรด / ขายคืน
Space	เรียกคลื่นถัดไปทันที (ได้โบนัสแรงศรัทธาตามเวลาที่เหลือ)
F	เร่งความเร็วเกม วน ×1 → ×2 → ×4
P / Esc · M · S	หยุดชั่วคราว · เปิด/ปิดเสียง · หน้าตั้งค่าเสียง

2.3 เครื่องราง (ป้อม)

ตารางที่ 3 เครื่องรางทั้ง 4 ชนิด

เครื่องราง	ราคา	ความสามารถ
ตะกรุด	50	ยิงเป้าหมายเดียว แรงและเร็ว แต่ยิงผีที่ลอยอยู่ไม่ได้
ระฆัง	150	สั่นสะเทือนเป็นวงกว้าง ตีได้พร้อมกันสูงสุด 5 / 7 / 9 ตนตามระดับ โดนผีลอยได้
สายสิญจน์	75	ไม่ทำความเสียหาย แต่ทำให้ผีในรัศมีเดินช้าลง
น้ำมนต์	140	สาดขวดไปตกเป็นแอ่ง ทำความเสียหายต่อเนื่องกับทุกตนในแอ่ง โดนผีลอยได้

ยิ่งมีเครื่องรางชนิดเดียวกันมาก ตัวถัดไปยิ่งแพงขึ้น (6–15% ต่อตัวตามระดับความยาก) ราคาในร้านจะเปลี่ยนเป็นสีส้มให้เห็น กลไกนี้ทำให้ผู้เล่นต้องผสมเครื่องรางหลายชนิดและวางแผนอัปเกรดแทนการสร้างชนิดเดียวเต็มสนาม



ภาพที่ 3 กดเลือกเครื่องรางซ้ำเพื่อดูรายละเอียดก่อนซื้อ



ภาพที่ 4 แผงข้อมูลเครื่องรางที่วางแล้ว อัปเกรด/ขายได้

2.4 วิญญาณ (ศัตรู)

ตารางที่ 4 วิญญาณทั้ง 5 ชนิด

วิญญาณ	พฤติกรรมพิเศษ
ผีเปรต	เดินตามเส้นทางตามปกติ เป็นศัตรูพื้นฐาน
ผีกระสือ	**บินตรง**เข้าพระอุโบสถ ไม่สนเส้นทาง ตะกรุดยิงไม่ได้
ผีปอบ	เลือดหนาและ**ฟื้นเลือดเอง**ทีละน้อย
ผีกองกอย	เมื่อตายจะ**แตกร่าง**เป็นเปรตน้อย 2 ตน
แม่นาค (บอส)	มีเกราะ **สาปเครื่องราง**รอบตัวให้หยุดทำงาน เรียกลูกสมุน และ**คลั่ง** เมื่อเลือดต่ำกว่า 30% (เร็วขึ้น 50%)



ภาพที่ 5 คลื่นสุดท้าย แม่นาคปรากฏตัวพร้อมแถบเลือดบอส

2.5 ด่านและระดับความยาก

ตารางที่ 5 ด่านทั้ง 3 ด่าน

ด่าน	ชื่อ	จุดเด่น
1	วัดร้างกลางป่า	ทางเดียวคดเคี้ยว เหมาะสำหรับเริ่มต้น
2	ท่าน้ำริมคลอง	มีคลองกั้นกลาง วางเครื่องรางบนน้ำไม่ได้ ผีกระสือบินลัดข้ามน้ำ
3	ลานเจดีย์สองป่าช้า	ผีมาจาก 2 จุดเกิดสลับกัน แล้วมารวมกันกลางลาน

ตารางที่ 6 ระดับความยาก

ความยาก	ศรัทธาเริ่มต้น	ดวงวิญญาณ	ราคาเพิ่มต่อตัวซ้ำ	ตัวคูณคะแนน
ง่าย	450	30	0%	×0.8
ปกติ	320	20	6%	×1.0
ยาก	300	10	12%	×1.6
ฝันร้าย	400	1 (ผีเข้าวัดตนเดียวก็แพ้)	15%	×2.5

นอกจากนี้ ทุกคลื่นผีจะเดินเร็วขึ้น 3% และค่าหัวผีจะลดลง 2.5% ต่อคลื่น เพื่อให้ช่วงท้ายเกมยังท้าทายและเงินไม่ล้นมือ



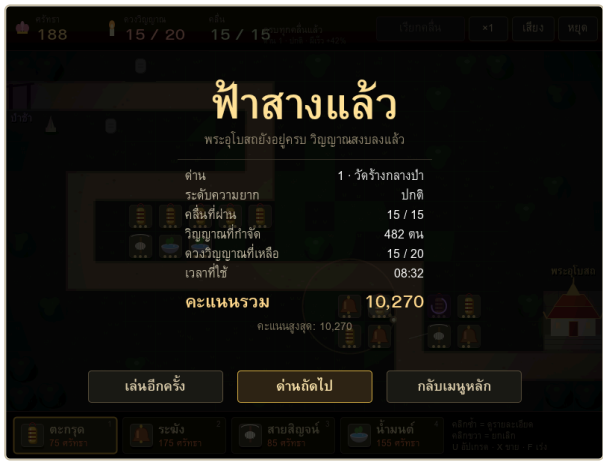
ภาพที่ 6 ด่าน 2 ท่าน้ำริมคลอง



ภาพที่ 7 ด่าน 3 โหมดฝันร้าย (หน้าจอมีโทนแดง)



ภาพที่ 8 ระหว่างเล่น เปิดความเร็ว ×4



ภาพที่ 9 หน้าสรุปผลเมื่อชนะ และบันทึกสถิติใหม่

บทที่ 3 การออกแบบโปรแกรม

3.1 โครงสร้างแพ็กเกจ

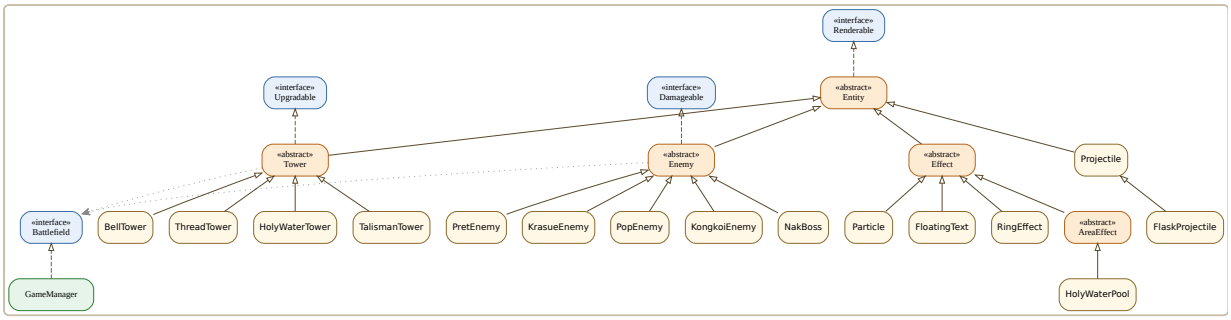
ซอร์สโค้ดทั้งหมดมี 56 ไฟล์ ประมาณ 4,800 บรรทัด แบ่งเป็น 7 แพ็กเกจตามหน้าที่ เพื่อให้แต่ละส่วนรับผิดชอบงานของตัวเองชัดเจน

ตารางที่ 7 แพ็กเกจและหน้าที่

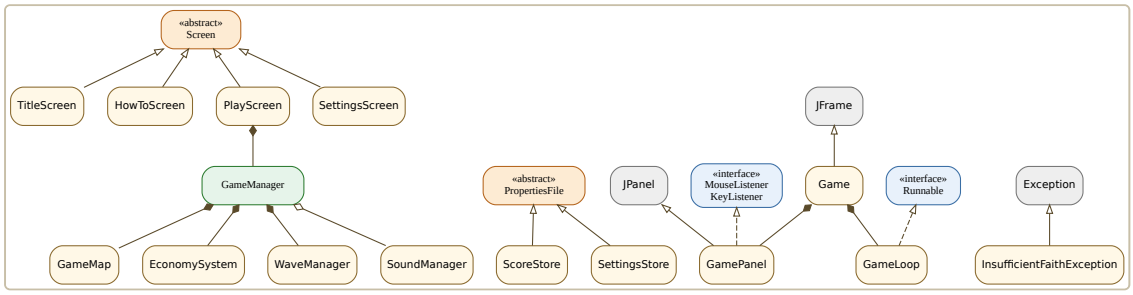
แพ็กเกจ	คลาสหลัก	หน้าที่
`core`	Game, GamePanel, GameLoop, GameManager, Battlefield, Setting	หน้าต่างเกม วงจรเกม และตัวควบคุมกติกาหลัก
`screen`	Screen → TitleScreen, HowToScreen, PlayScreen, SettingsScreen	หน้าจอแต่ละหน้าสลับกันแสดงผล
`entity`	Entity, Tower, Enemy, Projectile และคลาสลูก, TowerType, EnemyType	วัตถุทุกอย่างบนสนาม
`effect`	Effect, AreaEffect, Particle, FloatingText, RingEffect, HolyWaterPool	เอฟเฟกต์ภาพและพื้นที่ทำความเสียหาย
`system`	GameMap, Pathfinder, WaveManager, EconomySystem, SoundManager, PropertiesFile	ระบบย่อย: แผนที่ เส้นทาง คลื่น เงิน เสียง ไฟล์
`ui`	HUD, Button, Slider, Fonts, FieldRenderer	ส่วนติดต่อผู้ใช้และการวาดฉาก
`util`	Vec2<T>, interface ต่าง ๆ, enum Level/Difficulty/GameState/Sfx, Exception	ชนิดข้อมูลกลางที่ใช้ร่วมกัน

3.2 แผนภาพคลาส

ภาพที่ 10 แสดงลำดับชั้นการสืบทอดของวัตถุในเกม เส้นทึบหัวลูกศรสามเหลี่ยมคือการสืบทอด (extends) เส้นประคือการ implement interface ส่วนภาพที่ 11 แสดงหน้าจอ ระบบไฟล์ และความสัมพันธ์แบบ Composition ของ `GameManager`



ภาพที่ 10 แผนภาพคลาส: ลำดับชั้นของ Entity, Tower, Enemy และ Effect



ภาพที่ 11 แผนภาพคลาส: หน้าจอ ระบบไฟล์ และส่วนประกอบของ GameManager

3.3 วงจรการทำงานของเกม (Game Loop)

เกมแยกการทำงานเป็น 2 เธรด เธรด `game-loop` คำนวณสถานะเกมประมาณ 60 ครั้งต่อวินาที ส่วนเธรดของ Swing (Event Dispatch Thread) ทำหน้าที่วาดภาพและรับเมาส์/คีย์บอร์ด ทั้งสองเธรดใช้ตัวล็อกเดียวกัน (`synchronized`) จึงไม่เกิดกรณีวาดภาพขณะที่ข้อมูลกำลังเปลี่ยน ในแต่ละรอบ `GameManager.step()` จะทำงานตามลำดับดังนี้

1. `WaveManager` ปล่อยวิญญาณตัวถัดไปตามจังหวะของคลื่น
2. เครื่องรางทุกตัว `tick()` แล้ว `tryAttack()` แต่ละชนิดโจมตีคนละแบบ
3. กระสุนและแอ่งน้ำมนต์เคลื่อนที่และทำความเสียหาย
4. วิญญาณทุกตัวเคลื่อนที่ ตรวจว่าถึงพระอุโบสถหรือไม่ แล้วนำตัวที่ตายออก พร้อมให้แรงศรัทธา
5. ตรวจเงื่อนไขแพ้ (ดวงวิญญาณหมด) หรือชนะ (ครบ 15 คลื่นและไม่มีศัตรูเหลือ)

เมื่อผู้เล่นกดเร่งความเร็ว ×2 หรือ ×4 เกมจะเรียก `step()` ซ้ำ 2 หรือ 4 ครั้งต่อเฟรม แทนที่จะคูณเวลาให้ยาวขึ้น วิธีนี้ทำให้ผลลัพธ์เหมือนเล่นความเร็วปกติทุกประการ กระสุนไม่ทะลุเป้าแม้เร่งความเร็วสูง

core/GameManager.java

```
private void step(double dt) {
    waves.update(dt, this);
    for (Tower t : towers) {
        t.tick(dt);
        t.tryAttack(this);          // Polymorphism: ป้อมแต่ละชนิดโจมตีคนละแบบ
    }
    ...
    for (Enemy e : enemies) {
        e.tick(dt);                 // Polymorphism: ผีแต่ละชนิดเคลื่อนที่คนละแบบ
        e.interact(this, dt);
        if (e.hasReachedGoal() && e.isAlive()) onLeak(e);
    }
    ...
}
```

บทที่ 4 หลักการเขียนโปรแกรมเชิงวัตถุที่ใช้

ตารางที่ 8 สรุปหลักการ OOP และตำแหน่งในโค้ด

หลักการ	ตัวอย่างในโค้ด
Encapsulation	`EconomySystem` ซ่อนยอดเงินเป็น private แก้ได้ผ่าน `spend()` / `earn()` ที่ตรวจสอบแล้วเท่านั้น
Inheritance	ป้อม 4 ชนิดสืบทอด `Tower` · ผี 5 ชนิดสืบทอด `Enemy` · 4 หน้าจอสืบทอด `Screen` · `ScoreStore`/`SettingsStore` สืบทอด `PropertiesFile`
Polymorphism	`attack()`, `move()`, `onDeath()`, `interact()`, `predictPosition()` ถูก override ในคลาสลูก
Abstraction	abstract class 7 คลาส: Entity, Tower, Enemy, Effect, AreaEffect, Screen, PropertiesFile
Interface	`Renderable`, `Damageable`, `Upgradable` (มี default method), `Battlefield`
Composition	`GameManager` ประกอบจาก GameMap + EconomySystem + WaveManager + SoundManager
Overloading	`distanceTo(x, y)` / `distanceTo(Entity)` · constructor ของ `Difficulty` · `getHpScale()` / `getHpScale(Level)`
Enum	`TowerType`/`EnemyType` มี method `create()` ของแต่ละค่า · `Level`, `Difficulty` มี field และ constructor
Generics	`Vec2<T extends Number>` ใช้ได้ทั้งพิกัดช่อง (Integer) และพิกเซล (Double)
Exception	`InsufficientFaithException` โยนเมื่อแรงศรัทธาไม่พอ
Thread	`GameLoop implements Runnable` แยกเธรดคำนวณจากเธรดวาดภาพ
File I/O	อ่านแผนที่จาก resources · บันทึก `highscore.txt` และ `settings.txt`

4.1 Encapsulation และ Custom Exception

ยอดแรงศรัทธาเป็นข้อมูลสำคัญที่ไม่ควรถูกแก้จากที่ใดก็ได้ จึงซ่อนไว้เป็น `private` ในคลาส `EconomySystem` การใช้เงินต้องผ่านเมธอด `spend()` ซึ่งตรวจสอบก่อนเสมอ ถ้าเงินไม่พอจะโยน `InsufficientFaithException` ที่สร้างขึ้นเอง

ผู้เรียกจึงต้องจัดการกรณีนี้ (checked exception) และแสดงข้อความแจ้งผู้เล่นได้ถูกต้อง

system/EconomySystem.java

```
public class EconomySystem {
    private int faith;
    private int totalEarned;

    public void spend(int amount) throws InsufficientFaithException {
        if (amount > faith) throw new InsufficientFaithException(amount, faith);
        faith -= amount;
    }
    public void earn(int amount) {
        if (amount <= 0) return;
        faith += amount;
        totalEarned += amount;
    }
    public int getFaith() { return faith; }
}
```

4.2 Inheritance และ Abstraction

`Tower` เป็น abstract class ที่รวมสิ่งที่ป้อมทุกชนิดมีเหมือนกัน เช่น รัศมี ความเสียหาย คูลดาวน์ การเลือกเป้าหมาย และการอัปเกรด แต่ประกาศเมธอด `attack()`, `canTargetFlying()` และ `renderBody()` เป็น abstract เพราะ “โจมตีอย่างไร” และ “หน้าตาเป็นอย่างไร” ขึ้นกับป้อมแต่ละชนิด คลาสลูกจึงเขียนเฉพาะส่วนที่ต่างเท่านั้น

entity/Tower.java

```
public abstract class Tower extends Entity implements Upgradable {
    protected double range, damage, fireRate;
    protected int level = 1;

    protected abstract void attack(Battlefield field);   // คลาสลูกต้องเขียนเอง
    public abstract boolean canTargetFlying();
    protected abstract void renderBody(Graphics2D g, double cx, double cy);

    /** ตรวจเงื่อนไขร่วมก่อน แล้วค่อยให้คลาสลูกโจมตี */
    public final void tryAttack(Battlefield field) {
        if (isDisabled() || cooldown > 0) return;
        attack(field);
    }
}
```

entity/BellTower.java

```
public class BellTower extends Tower {
    public int getMaxTargets() { return 3 + level * 2; }       // 5 / 7 / 9 ตน

    @Override protected void attack(Battlefield field) {
        List<Enemy> inRange = new ArrayList<>();
        for (Enemy e : field.getEnemies())
            if (e.isAlive() && distanceTo(e) <= range) inRange.add(e);
        inRange.sort((a, b) -> Double.compare(b.getProgress(), a.getProgress()));
        int hits = Math.min(getMaxTargets(), inRange.size());
        for (int i = 0; i < hits; i++) inRange.get(i).takeDamage(damage);
        if (hits > 0) resetCooldown();
    }
    @Override public boolean canTargetFlying() { return true; }
}
```

4.3 Polymorphism

`GameManager` เก็บศัตรูทุกชนิดไว้ใน `List<Enemy>` เดียวกันและเรียก `e.tick(dt)` เหมือนกันหมด แต่ผลลัพธ์ต่างกันตามชนิดจริงของวัตถุ เช่น ผีกระสือ override เมธอด `move()` ให้บินเป็นเส้นตรงเข้าพระอุโบสถแทนการเดินตามทาง และ override `predictPosition()` ให้ทำนายตำแหน่งในอนาคตเป็นเส้นตรงด้วย ทำให้น้ำมนต์เล็งดักหน้าผีกระสือได้แม่นโดยไม่ต้องรู้ว่าเป็นผีชนิดใด

entity/KrasueEnemy.java

```
public class KrasueEnemy extends Enemy {
    @Override protected void move(double dt) {
        double dx = goalX - x, dy = goalY - y;
        double dist = Math.hypot(dx, dy);
        double step = getSpeed() * dt;
        if (dist <= step) { reachedGoal = true; return; }
        x += dx / dist * step;
        y += dy / dist * step;
    }
    @Override public boolean isFlying() { return true; }
}
```

ผีชนิดอื่นใช้ Polymorphism ในลักษณะเดียวกัน ได้แก่ `PopEnemy` override `update()` เพื่อฟื้นเลือด, `KongkoiEnemy` override `onDeath()` เพื่อแตกร่าง และ `NakBoss` override `interact()` เพื่อสาปเครื่องรางและเรียกลูกสมุน

4.4 Interface และ Default Method

Interface `Upgradable` กำหนดสัญญาของ “สิ่งที่อัปเกรดและขายคืนได้” และมี default method `canUpgrade()` คลาสที่ implement ไม่ต้องเขียนเมธอดนี้ซ้ำ ส่วน interface `Battlefield` ทำให้ป้อมและผีรู้จักสนามรบผ่านเมธอดที่จำเป็นเท่านั้น ไม่ต้องผูกกับคลาส `GameManager` ตัวจริง (Dependency Inversion) จึงทดสอบและแก้ไขแยกกันได้ง่าย

util/Upgradable.java · core/Battlefield.java

```
public interface Upgradable {
    int  getLevel();
    int  getMaxLevel();
    int  getUpgradeCost();
    void upgrade();
    int  getSellValue();
    default boolean canUpgrade() { return getLevel() < getMaxLevel(); }
}

public interface Battlefield {
    List<Enemy> getEnemies();
    List<Tower> getTowers();
    void spawnEnemy(Enemy e);
    void spawnProjectile(Projectile p);
    void playSound(Sfx s);
}
```

4.5 Template Method

การอัปเกรดของป้อมทุกชนิดมีขั้นตอนร่วมกัน คือ ตรวจว่าอัปเกรดได้ บันทึกเงินที่ลงทุน และเพิ่มระดับ ขั้นตอนนี้เขียนไว้ใน `upgrade()` แบบ `final` ส่วนผลของการอัปเกรดแยกไว้ใน `applyUpgrade()` ให้คลาสลูก override ได้ เช่น น้ำมนต์เพิ่มความเสียหายและขนาดแอ่ง รูปแบบเดียวกันนี้ใช้กับ `Entity.tick()` → `update()` และ `Enemy.render()` → `renderBody()`

entity/Tower.java · entity/HolyWaterTower.java

```
@Override public final void upgrade() {      // ใน Tower
    if (!canUpgrade()) return;
    totalInvested += getUpgradeCost();
    level++;
    applyUpgrade();                           // ส่วนที่คลาสลูกเปลี่ยนได้
}

@Override protected void applyUpgrade() {     // ใน HolyWaterTower
    damage     *= 1.35;
    poolRadius += 4;
    range      *= 1.06;
}
```

4.6 Enum แบบ Factory, Generics และ Overloading

`TowerType` เป็น enum ที่แต่ละค่ามีข้อมูลของตัวเอง (ชื่อไทย ราคา รัศมี สี คำอธิบาย) และ override เมธอด `create()` เพื่อสร้างป้อมชนิดของตัวเอง ร้านค้าจึงสร้างป้อมได้ด้วย `type.create(col, row)` คำสั่งเดียว ไม่ต้องใช้ if-else หรือ switch ยาว ๆ และเมื่อเพิ่มป้อมใหม่ก็เพิ่มแค่ค่าใน enum

entity/TowerType.java

```
public enum TowerType {
    TALISMAN("ตะกรุด", 50, 120, 0xD9B24A, "ยิงเป้าหมายเดียว แรงและเร็ว ...") {
        @Override public Tower create(int col, int row) { return new
TalismanTower(col, row); }
    },
    BELL("ระฆัง", 150, 100, 0xE0955C, "สั่นสะเทือนเป็นวงกว้าง ...") {
        @Override public Tower create(int col, int row) { return new BellTower(col,
row); }
    },
    ...;
    public abstract Tower create(int col, int row);
}
```

util/Vec2.java · entity/Entity.java

```
public class Vec2<T extends Number> {          // Generics
    private T x, y;
    public Vec2(T x, T y) { this.x = x; this.y = y; }
    public T getX() { return x; }
}

// Overloading ใน Entity: วัดระยะได้ทั้งจากพิกัดและจาก Entity อื่น
public double distanceTo(double px, double py) { return Math.hypot(x - px, y - py);
}
public double distanceTo(Entity other)         { return distanceTo(other.x,
other.y); }
```

4.7 Thread และ File I/O

`GameLoop` implement `Runnable` และทำงานในเธรดของตัวเอง จำกัดความเร็วที่ 60 เฟรมต่อวินาที และจำกัดค่า `dt` ไม่ให้เกิน 0.05 วินาที เพื่อไม่ให้เกมกระโดดเมื่อเครื่องสะดุด ส่วนการอ่าน-เขียนไฟล์รวมไว้ที่ abstract class `PropertiesFile` ทำให้ `ScoreStore` (คะแนนสูงสุด) และ `SettingsStore` (ระดับเสียง) สืบทอดไปใช้ได้โดยไม่ต้องเขียนโค้ดอ่านไฟล์ซ้ำ

system/PropertiesFile.java

```
public abstract class PropertiesFile {
    protected final Properties props = new Properties();
    private void load() {
        if (!Files.exists(file)) return;
        try (Reader r = Files.newBufferedReader(file, StandardCharsets.UTF_8)) {
            props.load(r);
        } catch (IOException e) {
            System.out.println("อ่านไฟล์ไม่ได้: " + e.getMessage());
        }
    }
    protected void save() { ... props.store(w, header); ... }
}
public class ScoreStore    extends PropertiesFile { ... }   // highscore.txt
public class SettingsStore extends PropertiesFile { ... }   // settings.txt
```

บทที่ 5 อัลกอริทึมและเทคนิคที่น่าสนใจ

5.1 หาเส้นทางด้วย BFS

แผนที่แต่ละด่านเก็บเป็นไฟล์ข้อความขนาด 20 × 12 ตัวอักษร เช่น `#` คือทางเดิน `S` คือป่าช้า `E` คือพระอุโบสถ เมื่อโหลดด่าน `Pathfinder` จะหาเส้นทางที่สั้นที่สุดจากป่าช้าแต่ละแห่งไปยังพระอุโบสถด้วย Breadth-First Search เพียงครั้งเดียว การเพิ่มด่านใหม่จึงทำได้ง่ายมาก คือวาดแผนที่ในไฟล์ข้อความแล้วเพิ่มค่าใน enum `Level` อีก 1 บรรทัด ไม่ต้องกำหนดเส้นทางเอง

system/Pathfinder.java (ย่อ)

```
queue.add(startId);
visited[startId] = true;
while (!queue.isEmpty()) {
    int cur = queue.poll();
    if (cur == goalId) break;
    for (int i = 0; i < 4; i++) {                  // ขวา ซ้าย ล่าง บน
        int nx = cur % w + DX[i], ny = cur / w + DY[i];
        if (!inBounds(nx, ny) || !map.isWalkable(nx, ny)) continue;
        int nid = ny * w + nx;
        if (visited[nid]) continue;
        visited[nid] = true;
        cameFrom[nid] = cur;                       // จำว่ามาจากช่องไหน
        queue.add(nid);
    }
}
// ย้อนรอยจาก goal กลับไป start เพื่อได้เส้นทาง
```

5.2 การเล็งดักหน้าของน้ำมนต์

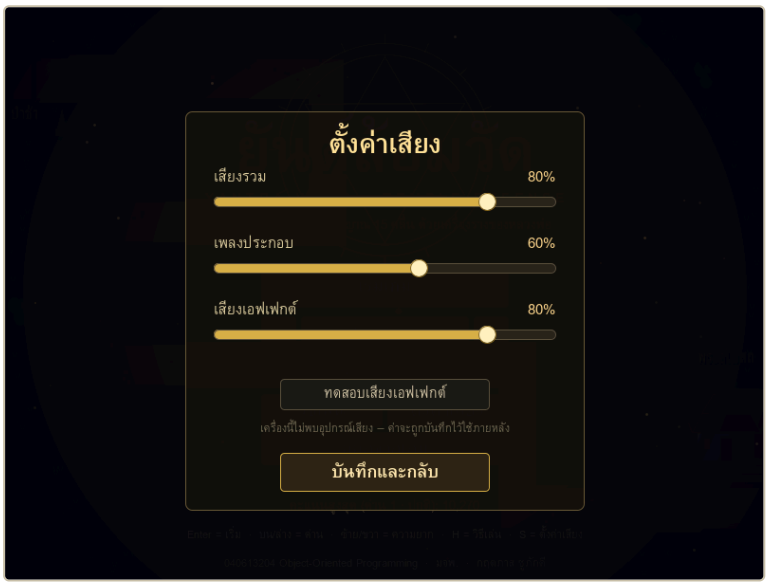
ขวดน้ำมนต์ลอยไปช้ากว่ากระสุนทั่วไป ถ้าเล็งตำแหน่งปัจจุบันของผีจะพลาดเสมอ จึงคำนวณเวลาบินของขวด แล้วถามผีว่า “อีก t วินาทีจะอยู่ที่ไหน” ผ่านเมธอด `predictPosition(t)` ทำซ้ำ 2 รอบเพื่อให้แม่นขึ้น ผีที่เดินตามทางจะทำนายตามเส้นทาง ส่วนผีกระสือทำนายเป็นเส้นตรง (Polymorphism)

entity/HolyWaterTower.java

```
double t = distanceTo(target) / FlaskProjectile.SPEED;
double[] aim = target.predictPosition(t);
t = Math.hypot(aim[0] - x, aim[1] - y) / FlaskProjectile.SPEED;
aim = target.predictPosition(t);
field.spawnProjectile(new FlaskProjectile(x, y - 6, aim[0], aim[1], damage,
poolRadius, poolDuration));
```

5.3 สังเคราะห์เสียงด้วยโค้ด

เกมไม่มีไฟล์เสียงเลย `SoundManager` สร้างคลื่นไซน์ที่มีความถี่และการลดระดับเสียงต่างกันแล้วแปลงเป็นข้อมูล PCM 16 บิต เช่น เสียงระฆังใช้คลื่นความถี่ 523, 1,445 และ 2,830 Hz ซ้อนกันให้ได้เสียงกังวาน ส่วนเพลงประกอบเป็นทำนองแนวระนาด ใช้บันไดเสียงเพนทาโทนิกแบบไทย ยาว 16 วินาทีและเล่นวนซ้ำ ผู้เล่นปรับระดับเสียงรวม เพลง และเอฟเฟกต์ได้จากหน้าตั้งค่า



ภาพที่ 12 หน้าตั้งค่าเสียง ค่าที่ตั้งจะบันทึกลง settings.txt

บทที่ 6 การทดสอบและปรับสมดุลเกม

6.1 วิธีทดสอบด้วยบอท

การปรับสมดุลเกมด้วยการเล่นเองอย่างเดียวใช้เวลามากและวัดผลได้ไม่แน่นอน ผู้จัดทำจึงเขียนบอทที่ควบคุมเกมผ่านคำสั่งเดียวกับผู้เล่นเท่านั้น (เลือกเครื่องราง คลิกช่อง อัปเกรด เรียกคลื่น) ไม่มีการแก้เงินหรือกติกา และรันเกมแบบไม่เปิดหน้าต่าง (headless) ได้เร็วกว่าเวลาจริงหลายร้อยเท่า บอทมี 2 แบบ คือ

- **บอทฮิวริสติก** เลือกช่องที่ครอบคลุมทางเดินมากที่สุดตามลำดับป้อมที่กำหนด ใช้วัดว่าผู้เล่นทั่วไปผ่านแต่ละระดับได้หรือไม่
- **บอท Genetic Algorithm** สุ่มแผนการเล่น (ลำดับการวางและอัปเกรด) แล้ววิวัฒน์ข้ามรุ่นจนพบแผนที่ชนะ ใช้แทน “ผู้เล่นที่เก่งที่สุด”

6.2 ผลการทดสอบ

ตารางที่ 9 ผลบอทฮิวริสติก (ดวงวิญญาณที่เหลือเมื่อจบเกม)

ด่าน	ง่าย (จาก 30)	ปกติ (จาก 20)	ยาก	ฝันร้าย
1	ชนะ 30	ชนะ 20	แพ้ คลื่น 7	แพ้ คลื่น 3
2	ชนะ 30	ชนะ 19	แพ้ คลื่น 6	แพ้ คลื่น 4
3	ชนะ 29	ชนะ 14	แพ้ คลื่น 6	แพ้ คลื่น 4

ระดับยากและฝันร้ายตั้งใจให้ยากกว่าที่บอทธรรมดาจะผ่านได้ จึงปรับด้วยบอท Genetic Algorithm แทน โดยค้นหาค่าเลือดผีสูงสุดที่บอทยังชนะได้แล้วตั้งค่าไว้ตรงนั้นแยกรายด่าน ผลคือบอทที่ดีที่สุดชนะแบบเฉียดฉิว และถ้าเพิ่มเลือดผีอีกเพียง 4% บอทจะแพ้

ตารางที่ 10 ค่าที่จูนสำหรับระดับยากและฝันร้าย (ตัวคูณเลือดผีเพิ่มเติมรายด่าน)

ด่าน	ยาก	ผลบอท GA	ฝันร้าย	ผลบอท GA
1	×1.16	ชนะ เหลือ 2/10	×1.32	ชนะ ไม่มีผีเข้าวัด
2	×1.32	ชนะ เหลือ 1/10	×1.52	ชนะ ไม่มีผีเข้าวัด
3	×1.04	ชนะ เหลือ 1/10	×1.08	ชนะ ไม่มีผีเข้าวัด

ค่าตัวคูณรายด่านนี้เก็บไว้ใน enum `Difficulty` ผ่าน constructor แบบที่สอง (Overloading) และเรียกใช้ผ่าน `getHpScale(Level)`

6.3 สิ่งที่ปรับจากผลการทดสอบ

ตารางที่ 11 การปรับสมดุลสำคัญ

ปัญหาที่พบ	วิธีแก้
วางระฆังอย่างเดียวก็ชนะ เพราะตีรอบด้านและแรง	ขึ้นราคาเป็นแพงที่สุด (150) และจำกัดจำนวนเป้าที่ตีได้ 5/7/9 ตน
สร้างป้อมชนิดเดียวกันเต็มสนาม (kill zone)	ราคาเพิ่มขึ้นตามจำนวนป้อมชนิดเดิม 6–15% ต่อตัว
เงินล้นมือช่วงท้ายเกม	ค่าหัวผีลดลง 2.5% ต่อคลื่น
ผีกระสือบินข้ามแนวป้องกันได้ง่ายเกินไป	จำกัดจำนวนต่อคลื่น และให้น้ำมนต์เล็งดักหน้าเป็นเส้นตรงได้
น้ำมนต์และการอัปเกรดสายสิญจน์แรงเกินไป	ลดความเสียหาย เพิ่มราคาน้ำมนต์เป็น 140 และลดผลการอัปเกรดสายสิญจน์
คลื่นท้าย ๆ ไม่กดดัน	ผีเดินเร็วขึ้น 3% ทุกคลื่น และแม่นาคคลั่งเมื่อเลือดต่ำ

บทที่ 7 ปัญหาที่พบและแนวทางแก้ไข

ตารางที่ 12 ปัญหาทางเทคนิค

ปัญหา	สาเหตุ	วิธีแก้
เพื่อนเปิดเกมไม่ได้ ขึ้น `UnsupportedClassVersionError`	jar คอมไพล์ด้วย Java 17 แต่เครื่องที่รันมี Java 8	คอมไพล์ jar ด้วย `--release 8` และเลี่ยงเมธอดของ Java 11 เช่น `isBlank()`
ภาพไม่คมเมื่อรันนอก NetBeans	Java 8 ไม่รองรับการขยายหน้าจอ (DPI scaling) 125–150%	`run.bat` ค้นหา Java 9 ขึ้นไปในเครื่องให้อัตโนมัติก่อน
สัญลักษณ์บางตัวแสดงเป็นกล่องสี่เหลี่ยม	ฟอนต์ภาษาไทยบางเครื่องไม่มีอักขระพิเศษ	วาดเป็นรูปทรงแทน และเลือกฟอนต์ไทยที่มีในเครื่องตามลำดับ
ภาพกระตุกหรือข้อมูลชนกัน	เธรดคำนวณกับเธรดวาดภาพเข้าถึงข้อมูลพร้อมกัน	ใช้ `synchronized` กับตัวล็อกเดียวกันใน `GamePanel`
เร่งความเร็วแล้วกระสุนทะลุเป้า	ถ้าคูณ `dt` ให้ใหญ่ วัตถุจะกระโดดข้ามกัน	เรียก `step()` หลายครั้งต่อเฟรมแทน

บทที่ 8 สรุปผล

8.1 สรุปผลการดำเนินงาน

โครงงานนี้พัฒนาเกม “ยันต์ล้อมวัด” ได้ครบตามขอบเขต มี 3 ด่าน 4 ระดับความยาก ป้อม 4 ชนิด ศัตรู 5 ชนิด ระบบอัปเกรด ระบบเสียง และระบบบันทึกคะแนน เกมเล่นได้จริงทั้งบน NetBeans และจากไฟล์ jar ด้านการเขียนโปรแกรมได้นำหลักการเชิงวัตถุทั้ง 4 ข้อมาใช้อย่างเป็นรูปธรรม พร้อมทั้ง Interface, Enum, Generics, Exception, Thread และ File I/O ผลของการออกแบบที่ดีเห็นได้ชัดเมื่อต้องเพิ่มด่านหรือปรับสมดุลภายหลัง ซึ่งแก้เพียงไม่กี่จุดโดยไม่กระทบส่วนอื่นของโปรแกรม

8.2 สิ่งที่ได้เรียนรู้

- การแยกส่วนที่ “เหมือนกัน” ไว้ในคลาสแม่ และส่วนที่ “ต่างกัน” ไว้ในคลาสลูก ทำให้โค้ดสั้นและเพิ่มชนิดใหม่ได้ง่าย
- การให้วัตถุรู้จักกันผ่าน interface ลดการผูกมัดระหว่างคลาส ทำให้แก้ไขและทดสอบแยกส่วนได้
- การทดสอบด้วยบอทอัตโนมัติช่วยให้ปรับสมดุลเกมโดยอาศัยข้อมูลจริงแทนการเดา
- การส่งโปรแกรมให้ผู้อื่นใช้ต้องคำนึงถึงสภาพแวดล้อมของเครื่องปลายทาง เช่น รุ่นของ Java

8.3 แนวทางพัฒนาต่อ

- เพิ่มด่าน เครื่องราง และศัตรูชนิดใหม่ (โครงสร้างปัจจุบันรองรับอยู่แล้ว)
- เพิ่มโหมดดูบอทเล่นภายในเกม สำหรับสาธิตหรือสอนกลยุทธ์
- บันทึกและเล่นต่อจากเกมที่ค้างไว้ และเพิ่มตารางคะแนนออนไลน์
- ทำตัวติดตั้งแบบ .exe ที่รวม Java ไว้ในตัว ให้ผู้เล่นไม่ต้องติดตั้ง Java เอง

ภาคผนวก วิธีติดตั้งและรันเกม

ตารางที่ 13 ช่องทางการรันเกม

วิธี	ขั้นตอน
เปิดใน NetBeans	File → Open Project เลือกโฟลเดอร์ `YantraWard` แล้วกด Run Project (F6) ถ้าถาม Main Class ให้เลือก `yantra.Main`
เล่นทันที	ดับเบิลคลิก `run.bat` (ต้องมี Java 8 ขึ้นไป แนะนำ Java 17 ขึ้นไป)
ซอร์สโค้ด	https://github.com/Krittapas-01/YantraWard

ตารางที่ 14 ไฟล์สำคัญในโปรเจกต์

ไฟล์ / โฟลเดอร์	รายละเอียด
`src/main/java/yantra/`	ซอร์สโค้ด Java ทั้งหมด 56 ไฟล์
`src/main/resources/maps/`	แผนที่ด่าน 1–3 แบบไฟล์ข้อความ
`pom.xml`	การตั้งค่า Maven
`YantraWard.jar`, `run.bat`	ไฟล์เกมที่ build แล้ว และสคริปต์เปิดเกม
`README.md`, `docs/`	คู่มือ ภาพหน้าจอ และวิดีโอตัวอย่างการเล่น